

3. We had to follow lengthy procurement processes to purchase new servers

4. We had to patch and maintain the servers

Cloud and virtualisation provided the level of flexibility that physical servers lacked. We didn't have to follow lengthy procurement processes, who "owns the server," why only that team has "exclusive access to that powerful server," etc. Procurement processes for physical machines became obsolete with virtual machines (VMs) and cloud. The way we architected also changed. For example, instead of 'scaling up' by adding more CPUs or memory to physical servers, we started 'scaling out' by adding more machines as needed in the cloud. This model gave us the flexibility of operational expenses-based (OPEX-based) economics model. If any of the VMs went down, we got new VMs spawned in minutes. In short, we started treating servers as 'cattle' and not 'pets.'

However, cloud and virtualisation also came with their own problems and still have many limitations. Users are still spending a lot of time managing them, for example, bringing VMs up and down based on their need. They have to architect for availability and fault-tolerance, size workloads, and manage capacity and utilisation. If they have dedicated VMs provisioned in the Cloud, they still have to pay for the reserved resources (even if it's just idle time). Hence, moving from CAPEX to OPEX is not enough; what users need is to "really only pay what they are using" and "really pay as they go." Serverless computing promises to address exactly this problem.

Another key aspect is agility. Businesses today need to be very agile. Technology complexity and infrastructure operations can-

not be used as an excuse for not delivering value at scale. Ideally, much of the engineering effort should be focused on delivering functionality that delivers the desired experience, and not on monitoring and managing the infrastructure that supports the scale requirements. This is where serverless shines.

## What is serverless?

Consider a chatbot for booking movie tickets—let's call it MovieBot. Any user can query about movies, book tickets or cancel them in a conversational style; for example, "Is Dunkirk playing in Urvashi Theatre in Bangalore tonight?" in voice or text.

This solution requires three elements: a chat interface channel (like Skype or Facebook Messenger), a Natural Language Processor (NLP) to understand the user intents (book a ticket, ticket availability, cancellation, etc), and then access to a backend where transactions and data pertaining to movies are stored. The chat interface channels are universal and can be used for different kinds of bots. NLP can be implemented using technologies like AWS Lex or IBM Watson. The question is: How is the backend served? Would you set up a dedicated server (or a cluster of servers) and an API gateway, deploy load balancers, put in place identity and access control mechanisms, etc? That's costly and painful, right! That's where serverless technology can help.

The solution is to set up some compute capacity to process data from a database and also execute this logic in a language of choice. For example, if you are using AWS platform, you can use DynamoDB for the backend, write programming logic as Lambda functions, expose them through AWS API gateway with a load balancer.

This entire setup does not require you to provide any infrastructure or have any knowledge about underlying servers/VMs in the cloud. You can use a database of your choice for the backend. You can choose any programming language supported in AWS Lambda, including Java, Python, JavaScript and C#. There is no cost involved if there are no users of the MovieBot. If a blockbuster like *Bahubali* is released, there could be a huge surge in users accessing the MovieBot at the same time, and the setup would effortlessly scale (you have to pay for the calls, though). Phew! You essentially engineered a serverless application.

Serverless architectures refer to applications that significantly depend on third-party services (known as backend-as-a-service or BaaS) or on custom code that's run in ephemeral containers (function-as-a-service or FaaS).

**1. Backend-as-a-service.** Typically, databases (often NoSQL types) hold data and these could be accessed over the cloud. A service can be used to help access that backend. Such a backend service is referred to as BaaS.

**2. Function-as-a-service.** Code that processes requests (the programming logic written in your favourite programming language) could be run on containers that are spun and destroyed as needed. These are known as FaaS.

The word 'serverless' is misleading because it literally means there are 'no servers.' Actually, the word 'serverless' means "I don't care what a server is." In other words, serverless enables you to create applications without thinking about servers, that is, you can build and run applications or services with-